

软件过程改进方案设计文档

一、文档概述

本报告针对“校园集市”系统的原有软件过程（基于ISO/IEC 12207裁剪后的轻量级过程），结合《AI技术应用场景匹配报告》中识别的四个关键环节：需求分析与文档生成、UML建模、编码实现、测试用例生成，设计了一套融合AI技术的改进方案。方案包含过程重构、工具链选型、风险评估与应对策略，旨在构建适配智能化开发趋势的新型软件过程体系。

二、过程重构方案

原有软件过程包括：需求定义、架构定义、设计定义、实现与单元测试、集成与集成测试、系统测试与确认，以及配置管理、质量保证、项目管理等支持与管理活动。改进后，在关键环节引入AI工具与新增角色，调整活动流程，并定义AI输出物的质量标准与验证机制。

2.1 新增AI相关角色

角色名称	职责描述	所属阶段
AI需求分析师	负责设计需求提示词、运行LLM生成SRS初稿、审查并修正AI输出、维护需求可追溯性	需求分析
AI建模师	使用LLM辅助生成UML模型，验证模型与需求的一致性，修正模型缺陷	系统设计
AI开发师	使用AI编码工具进行开发，制定代码采纳标准，评估AI生成代码的质量与风险	编码开发
AI测试分析师	设计测试提示词，生成测试用例与脚本，评估测试采纳率，维护测试资产	系统测试与确认

2.2 活动流程调整

竞品分析与用户故事收集



AI需求分析师



生成SRS初稿



人工审查与修正SRS



需求评审与基线化



AI建模师

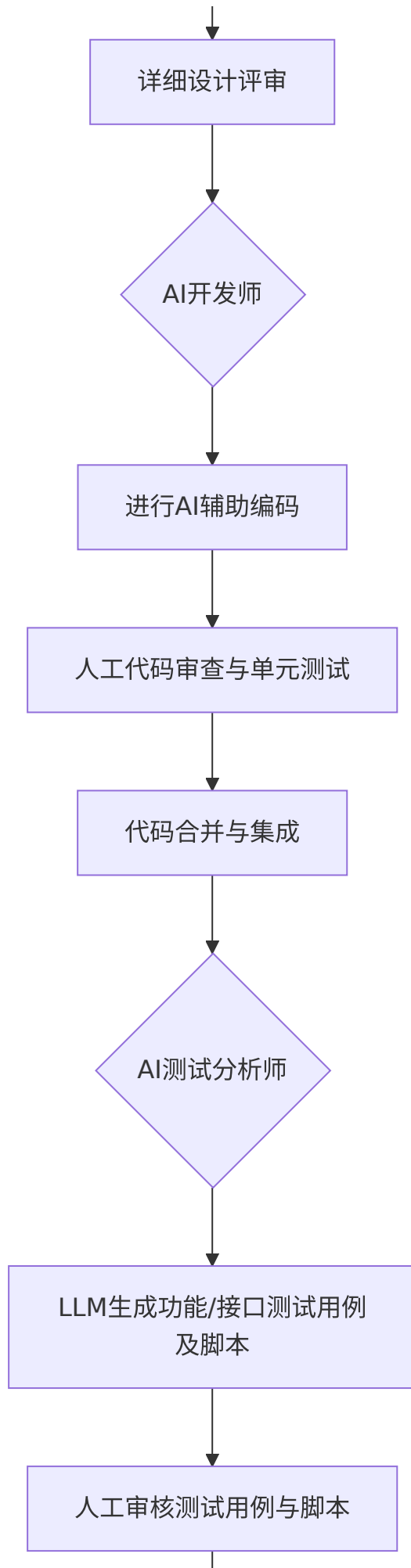


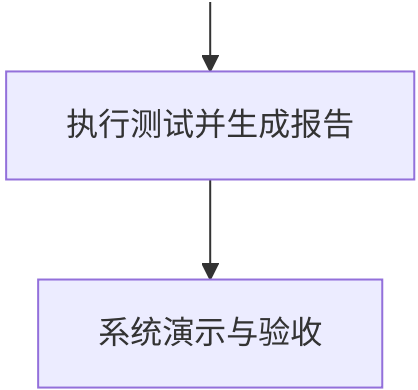
LLM生成用例图/类图/时序图



人工修正与完善UML模型







调整说明：

- 需求定义阶段：原为人工编写SRS，现改为AI生成初稿 + 人工审查修正，新增AI需求分析师角色。
- 设计定义阶段：原为手工绘制UML，现改为AI生成模型 + 人工修正，新增AI建模师角色。
- 实现与单元测试阶段：原为纯手工编码，现引入AI编码辅助。
- 系统测试与确认阶段：原为手工编写测试用例并执行，现引入AI生成测试用例与脚本，新增AI测试分析师角色。

2.3 AI输出物的质量标准与验证机制

AI输出物	质量标准	验证机制
SRS文档	完整性（覆盖所有用户故事）、内部一致性（无矛盾需求）、可追溯性（每条需求标注来源）、无歧义性	人工审查检查表 + 交叉评审；使用需求一致性验证工具
UML模型	与SRS需求一致、符合UML语法规则、类关系正确、方法签名准确	人工逐项核对需求追溯矩阵；使用模型验证工具
AI生成代码	功能正确性（通过单元测试）、代码风格符合规范、无安全漏洞、注释充分	静态分析工具；人工代码审查；单元测试覆盖率≥80%
测试用例与脚本	需求覆盖度≥95%、断言准确、脚本可执行、维护性良好	人工审核用例与需求追踪矩阵；运行生成的脚本并检查通过率

三、工具链选型

针对四个改进环境，推建以下AI工具或技术框架：

3.1 需求分析与文档生成

- **工具名称：**ReqInOne（基于GPT-3.5-turbo）
- **核心功能：**将非结构化的用户故事、会议记录转换为结构化SRS文档，支持需求分解、分类和可追溯性标注。
- **集成方式：**需求分析师将收集的用户故事整理为提示词输入，工具生成Markdown格式的SRS初稿，然后导入项目文档仓库（Git）。
- **预期效率提升：**SRS撰写时间缩短50%以上，需求遗漏率降低30%。

3.2 UML建模

- **工具名称：**Llama 3.1 70B + 提示工程模板
- **核心功能：**根据需求文本自动生成用例图、类图、时序图的PlantUML或Mermaid代码。
- **集成方式：**开发者将SRS章节粘贴到LLM聊天界面，获得模型代码后复制到Markdown文档中渲染。也可使用VS Code插件“PlantUML”辅助验证。
- **预期效率提升：**建模时间平均缩短60%，模型与需求一致性提升。

3.3 编码实现

- **工具名称：**Trae
- **核心功能：**代码自动补全、函数级代码生成、智能重构、代码解释、单元测试生成。
- **集成方式：**Trae 提供独立的AI原生IDE客户端，开发者可直接在IDE中编写代码，调用AI能力。
- **预期效率提升：**编码效率提升30%~40%，代码生成采纳率达40%左右，代码缺陷率降低20%。

3.4 测试用例与脚本自动生成

- **工具名称：**Trae
- **核心功能：**根据API文档与相关接口代码生成功能测试用例和接口测试脚本（Python/Postman格式），支持测试数据生成和自愈脚本修复。
- **集成方式：**AI IDE直接读取项目代码，生成测试用例表格和可执行脚本，由AI测试分析师进行检查与执行。
- **预期效率提升：**测试设计时间缩短50%，用例采纳率可达40%~50%，回归测试维护成本降低30%。

四、风险评估与应对策略

风险类别	具体风险描述	发生概率	应对策略
数据安全	需求文档、代码等敏感信息上传至公有云API，可能泄露	中	优先选用支持本地私有化部署的模型（如DeepSeek、Llama系列）；对API调用进行数据脱敏；签署数据保密协议
模型偏见	AI生成的需求或测试用例偏向常见场景，忽略边缘情况	中	人工审查阶段增加“边缘场景检查清单”；要求AI生成时明确提示“请同时列出可能的异常情况”；结合变异测试验证测试充分性
技术依赖	过度依赖特定AI工具，一旦工具停服或收费，开发受阻	高	选择开源或有多家替代品的工具；保留人工操作能力作为备份；制定工具切换预案
质量下降	AI生成代码存在逻辑错误或安全漏洞，直接使用导致缺陷	中	强制执行SonarQube自动扫描；代码入库前必须通过CI检查（测试覆盖率、静态分析）
合规风险	引用第三方代码片段但未遵守开源协议	低	培训开发者识别AI生成代码的许可证风险；使用代码扫描工具（如FOSSA）检测许可冲突；要求AI明确标注代码来源

参考文献

[1] 《AI4SE行业现状调查报告》（中国信通院，2026）

[2] 《智能化软件开发落地实践指南》（中国信通院，2024）

[3] T. Zhu, L. C. Cordeiro and Y. Sun, "ReqInOne: A Large Language Model-Based Agent for Software Requirements Specification Generation," 2025 IEEE 33rd International Requirements Engineering Conference (RE), Valencia, Spain, 2025, pp. 449-457.

[4] T. Eisenreich, N. Friedlaender and S. Wagner, "Leveraging Large Language Models for Use Case Model Generation from Software Requirements," 2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW), Seoul, Korea, Republic of, 2025, pp. 221-227.

[5] Jiang, Juyong, et al. "A survey on large language models for code generation." ACM Transactions on Software Engineering and Methodology 35.2 (2026): 1-72.

[6] Baqar, Mohammad, and Rajat Khanda. "The future of software testing: AI-powered test case generation and validation." *Intelligent Computing-Proceedings of the Computing Conference*. Cham: Springer Nature Switzerland, 2025.